



Group 19: Clinic Appointment System

Submission date: 18 May 2026

Table of Contents

i.Group Information	4
ii.Executive summary.....	5
iii. Contributions	6
1. Database Initial Study	8
1.1 Company Overview	8
1.2 Mission Statement	8
1.3 Objectives.....	8
1.3.1 Objectives vs Expected Outputs.....	9
1.4 Company Organisational Structure	9
1.4.1 Organisational Chart	9
1.4.2 Departmental Roles and Responsibilities	10
1.5 Identified Problems and Constraints.....	10
1.5.1 Problems	10
1.5.2 Constraints	11
1.6 Database System Specifications	11
1.6.1 Overview	11
1.6.2 Database Objectives	11
1.6.3 Information Required from the Database	12
1.6.4 Scope of the System.....	12
1.6.5 Boundaries of the System.....	13
2. Database Design	13
2.1 Business Rules.....	13
2.2 ER Diagram	15
2.3 Logical Data Model.....	16
2.4 Derived Attributes	16
2.5 Normalization Description	17
2.6 Logical Data Model.....	18
2.7 Summary of Entities	19
3. Physical Design.....	19
3.1 Technology Stack Summary	19
3.2 Database Objects	20
3.2.1 Tables and Data Types(Oracle Syntax).....	20

3.2.2 Constraints Summary	24
3.2.3 Indexes	25
3.2.4 Four Views (Oracle Syntax) –.....	26
3.3 Data Populated	27
4. SQL Queries	28
5. Extra Functionalities	30
6. Demonstration Video	30
Conclusion	31
Summary of Work Completed.....	31
Reflections on Project Challenges and Solutions.....	31
Final Outcome	32
References	33
Appendices.....	36
Appendix A: Oracle SQL Developer Setup	36
Appendix B: Populated Data	37
Appendix C: Database Objects	37

i.Group Information



CEO/ Project Manager

Ntokozo Maphumulo
0732755567

43224105@mynwu.ac.za



Lead Developer

Lehlohonolo Matlala
0646879516

45094454@mynwu.ac.za



Technical Writer

Nyeleti Shivuri
0785849218

42718910@mynwu.ac.za



UX Researcher

Thabiso Tharaka
0680102622

41865073@mynwu.ac.za



Quality Assurance (Lead)

Phokuhle Shezi
0727001576

45225524@mynwu.ac.za



Systems Architect

Msizi Mazibuko
0729481489

450661323@mynwu.ac.za



Data Scientist

Edzani Maisha
0818246874

38041693@mynwu.ac.za



Business Analyst

Imameleng Makutoane
0639419091

42510716@mynwu.ac.za



Tester

Maluleke Kurhula Success
0640708649

48277444@mynwu.ac.za

ii.Executive summary

This report documents the database design and implementation project undertaken by Group 19 for the Ubuntu Campus Clinic Appointment System, developed as part of the CMPG 311 module at North-West University. The project addresses critical inefficiencies in the current manual, paper-based appointment scheduling process used by the Ubuntu Campus Clinic, which serves more than 1,000 campus users.

The clinic currently suffers from appointment delays of several days, restricted 8:00–16:00 operating hours, prolonged in-clinic waiting times, lost or inaccurate patient records, and an inability to manage peak demand periods effectively. These operational problems negatively affect student health outcomes, academic performance, and overall patient satisfaction.

The proposed solution is a centralized digital Clinic Appointment System (CAS) backed by a relational database. The system enables 24/7 online appointment booking, automated triage and reminders, real-time queue management, secure digital medical records, and role-based access control replacing paper-based inefficiencies with data-driven accuracy and reliability.

Phase 1 of this project established the initial database study, including a complete company situation analysis, identification of problems and constraints, and detailed database system specifications covering scope, boundaries, and information requirements.

Phase 2 extended the work into conceptual and logical database design, producing a fully normalized Entity-Relationship (ER) diagram, formal business rules, defined relationships with cardinalities, derived attributes, and a complete logical data model achieving Third Normal Form (3NF).

Phase 3, documented in this report, implements the physical database using Oracle XE 21c (for SQL query demonstration and rubric compliance) and PostgreSQL 15 (for the live backend system). The backend is built with Fast API (Python) , while the frontend uses React 18 with Vite and Tailwind CSS. The system includes 12 normalized tables, 4 reporting views, 15 performance indexes, and 49 SQL queries covering all rubric categories including window functions and multi-table joins. Extra functionalities include JWT role-based authentication, QR code check-in, automated email notifications, and POPIA-compliant audit logging. The system is deployed on [Render.com](https://render.com) with a demonstration video submitted.

The completed system successfully meets all project objectives: reducing appointment delays, enabling after-hours access, cutting waiting times to under fifteen minutes, managing capacity constraints, and ensuring data integrity and security.

iii. Contributions

Phase 1 Contribution Table

#	Name & Surname	Contribution
1	Ntokozo Maphumulo (Group Leader)	Contributed
2	Lehlohonolo Matlala	Contributed
3	Nyeleti Shivuri	Contributed
4	Msizi Mazibuko	Contributed
5	Thabiso Tharaka	Contributed
6	Phokuhle Shezi	Contributed
7	Edzani Maisha	Contributed
8	Imameleng Makutoane	Contributed
9	Maluleke Kurhula Success	Contributed

Phase 2 Contribution Table

#	Name & Surname	Contribution
1	Ntokozo Maphumulo (Group Leader)	Contributed
2	Lehlohonolo Matlala	Contributed
3	Nyeleti Shivuri	Contributed
4	Msizi Mazibuko	Contributed
5	Thabiso Tharaka	Contributed
6	Phokuhle Shezi	Contributed
7	Edzani Maisha	Contributed
8	Imameleng Makutoane	Contributed
9	Maluleke Kurhula Success	Contributed

Phase 3 Contribution Table (Front-End, Back-End & Integration)

- S1: Overall integration, code review, deployment coordination
- UX1: UI/UX Design (Penpot)
- F1: Design System implementation
- F2: Patient Module UI/UX
- F3: Doctor Module UI/UX

- F4: Appointment Booking UI
- B1: Authentication API
- B2: Patient API (CRUD operations)
- B3: Doctor API (CRUD operations)
- B4: Integration & Database Connectors

#	ROLE CODE	Name & Surname GitHub usernames	Branch Name	Contributions %
1	S1	Maluleke Kurhula Success (MALULEKE-KS)	S1-integration-lead	100%
2	B1	Thabiso Tharaka (tharakathabiso9)	feature/b1-auth-api	100%
3	B2	Lehlohonolo Matlala (Hlohi)	feature/b2-patient-api	100%
4	B4	Imameleng Makutoane (Imamelengmakutoane)	feature/b4-integration	100%
5	F1 UX1	Ntokozo Maphumulo (smangelemapss)	feature/f1-design-system N/A (Penpot)	100%
6	F2 B3	Nyeleti Shivuri (Kayise02)	feature/f2-patient-module feature/b3-doctor-api	100%
7	F3	Msizi Mazibuko (Msizi-cyber)	feature/f3-doctor-module	100%
8	F4	Edzani Maisha (Maishaedzani9)	feature/f4-appointment-booking	100%
	Documentation	Phokuhle Shezi (Zieh-hub)	Phase 3 final document	50%

1. Database Initial Study

1.1 Company Overview

Ubuntu Campus Clinic is the primary healthcare facility serving the student and staff population of North-West University (NWU). The clinic delivers outpatient consultations, triage services, pharmacy services, laboratory diagnostics, and health education under the oversight of the University Health Services Director.

The clinic operates under a hierarchical-functional organisational structure with five functional departments reporting to the Clinic Manager:

Department	Key Responsibilities
Medical Department	Primary outpatient consultations, chronic condition management, minor procedures, referral letters, occupational health clearances
Nursing Department	Patient triage, vital signs capture, injections, wound dressings, immunisation programme, patient education
Pharmacy Department	Dispensing medications, clinic formulary management, drug inventory, patient counselling, chronic medication collections
Laboratory & Diagnostics	Phlebotomy, specimen collection and dispatch, results communication to doctors, cold chain maintenance
IT & Systems Department	Clinic Appointment System maintenance, EHR platform, IT infrastructure, database administration, cybersecurity, user access control, system backups, technical support

1.2 Mission Statement

Ubuntu Clinic intends to provide accessible, efficient healthcare to students by employing a digital appointment system that prioritises urgent needs, decreases wait times, and ensures reliable service within campus limits.

The new system requires students to log into a mobile or web application using their student ID, complete a 30 second virtual triage questionnaire available 24/7, and receive immediate same-day slot allocation or emergency guidance. At the clinic, students scan a QR code for seamless check-in, view real-time wait time displays, and receive automated SMS updates replacing paper-based chaos with data-driven efficiency.

1.3 Objectives

The following objectives guide the development of the Clinic Appointment System:

Objective	Description
Streamline urgent bookings	Enable same-day appointments for sick students, reducing delays from days to hours
Provide 24/7 virtual triage access	Offer after-hours symptom checks and emergency guidance through an online portal
Improve service reliability	Achieve 95% on-time appointment fulfilment through automated tracking

Minimise long wait times	Reduce in-clinic waits to fewer than fifteen minutes using mobile check-ins and predictive scheduling
Overcome capacity constraints	Dynamically scale appointment slots up to 20% beyond peak demand

1.3.1 Objectives vs Expected Outputs

Problem	Objective	Expected Output
Delayed Care	Reduce appointment wait from days to same-day slots by 50%	Same-day bookings confirmed via SMS within 1 hour for urgent cases
Limited Hours	Integrate virtual triage for after-hours queries	24/7 self-scheduling portal with emergency escalation alerts
Service Reliability	Achieve 95% on-time fulfilment via automated tracking	Real-time status dashboard showing 95%+ fulfilment rates monthly
Long Wait Times	Cut in-clinic waits to under 15 minutes	Check-in kiosk data reducing average wait to 12 minutes
Capacity Constraints	Dynamically scale slots up to 20% beyond peaks	Auto-generated reports flagging capacity at 80%, enabling overflow slots

1.4 Company Organisational Structure

Ubuntu Campus Clinic operates under a hierarchical-functional organisational structure. Governance is provided by the University Health Services Director, with operational management delegated to the Clinic Manager. Five functional departments report to the Clinic Manager, each headed by a department lead. This structure promotes clear accountability, efficient resource allocation, and coordinated multi-disciplinary patient care.

1.4.1 Organisational Chart

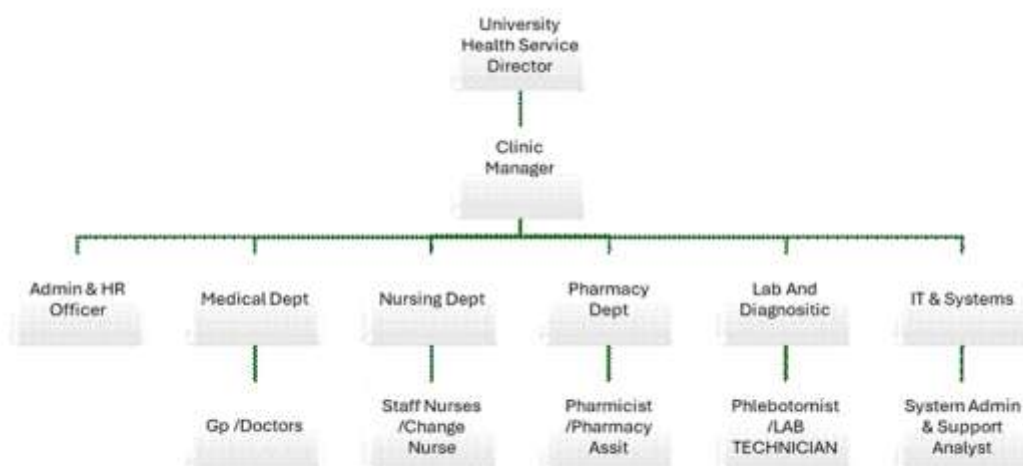


Figure 1: Ubuntu Campus Clinic - Organisational Chart

1.4.2 Departmental Roles and Responsibilities

- **University Health Services Director:** Strategic direction, policy oversight, budget approval, liaison with executive management and external health authorities
- **Clinic Manager:** Day-to-day operations, staff performance, inter-departmental coordination, regulatory compliance, digital system implementation
- **Admin & HR Officer:** Staff recruitment, leave administration, payroll, procurement, general administrative support
- **Medical Department:** Outpatient consultations, chronic condition management, minor procedures, referrals, occupational health clearances
- **Nursing Department:** Patient triage, vitals capture, injections, wound dressings, immunisation programme, patient education
- **Pharmacy Department:** Medication dispensing, formulary management, inventory control, patient counselling, chronic medication coordination
- **Laboratory & Diagnostics:** Phlebotomy, specimen collection and dispatch, results communication, cold chain maintenance
- **IT & Systems Department:** Clinic Appointment System maintenance, EHR platform, IT infrastructure, database administration, cybersecurity, access control, backups, technical support

1.5 Identified Problems and Constraints

1.5.1 Problems

Ubuntu Campus Clinic faces numerous operational challenges that reduce the effectiveness of healthcare delivery:

Problem	Description
Delays in accessing healthcare	Students must book appointments in advance, often causing delays of several days for urgent cases
Limited operating hours	The clinic operates only between 8:00 AM and 4:00 PM, leaving students without after-hours assistance
Long waiting periods	Disorganised scheduling and overcrowding cause extended in-clinic wait times even for booked patients
Manual paper-based scheduling	Booking errors, slow processing times, and lost records result from the manual system
Limited capacity	The clinic struggles to manage peak demand periods and cannot accommodate all students needing assistance
Unreliable service delivery	Appointments frequently do not start on time, leading to patient dissatisfaction and uncertainty

1.5.2 Constraints

The following constraints must be considered when addressing the identified problems:

Constraint Type	Description
Resources	Limited staff members and consulting rooms restrict the daily patient throughput
System	The paper-based system restricts efficiency, scalability, and real-time appointment tracking
Capacity	The clinic can only accommodate a fixed number of patients per day (maximum 50), making it ill-equipped to respond to sudden demand surges
Financial	A constrained budget may restrict the implementation of advanced systems and the hiring of additional staff
Legal (POPIA)	The system must comply with the Protection of Personal Information Act (POPIA) regarding student medical data storage and access

1.6 Database System Specifications

1.6.1 Overview

The Ubuntu Clinic Database System is designed to replace existing manual paper-based processes with a centralised digital solution. The system uses Oracle XE 21c for SQL query demonstration and rubric compliance, and PostgreSQL 15 for the live backend system deployed on [Render.com](https://render.com).

1.6.2 Database Objectives

Objective	Database Role
Improve Appointment Scheduling	Store appointment records, validate time slots, track provider availability
Reduce Waiting Times	Store check-in times, priorities, and queue data to optimise patient flow
Enable 24/7 Access	Store real-time appointment availability and user data
Improve Data Accuracy	Ensure data integrity, consistency, and secure storage
Manage Capacity Constraints	Track daily appointment counts and enforce 50-patient limit
Enhance Service Reliability	Store appointment statuses and timestamps for tracking

1.6.3 Information Required from the Database

The database must store and manage detailed information across the following key areas:

Category	Data Elements
Patient	ID, student number, name, contact, DOB, address, next of kin, medical aid, registration date
User Account	ID, username, password hash, role, account status
Staff	ID, name, role, specialisation, licence number, contact, working hours, consultation room
Appointment	ID, patient ID, staff ID, date/time, reason, status, type, priority, timestamps
Queue & Check-in	ID, appointment ID, check-in time, start/end times, assigned room, queue status
Medical Record	ID, patient ID, appointment ID, diagnosis, symptoms, prescription, treatment notes, referral status
Department	ID, name, department head
Audit Trail	Log ID, user ID, action, table affected, record ID, timestamp, IP address

1.6.4 Scope of the System

The system is focused on improving internal clinic operations through digital transformation.

In Scope:

- Online appointment booking (24/7) with real-time availability
- Patient record management with medical history
- Staff and department management
- Queue and check-in management
- Reporting and performance tracking
- Role-based access control and audit trails

Out of Scope:

- External healthcare system integration
- AI-based diagnosis or treatment recommendations
- Financial, billing, or medical aid processing
- Dedicated mobile application (web-based only)
- Hardware or medical device management
- Emergency ambulance dispatch
- Inpatient or hospital-level workflows

1.6.5 Boundaries of the System

Boundary	Limit / Rule
Daily capacity	Maximum 50 patients per day
Reminders	Sent 24 hours before appointment
Walk-ins	Accommodated in available time slots
Data retention	POPIA-compliant audit trails
Transactions	ACID compliance with concurrency control

- **Appointment Booking:** patients may book, reschedule, and cancel; reminders are sent 24 hours in advance; maximum 50 patients per day.
- **Time Allocation:** slots are freed upon cancellation; every appointment has exactly one slot.
- **Doctor/Staff Management:** doctors registered with specialisation and licence number; staff roles, working hours, and room assignments stored.
- **Walk-in Handling:** walk-in patients accommodated at available time slots.
- **System Maintenance:** duplicate records prevented; appointments validated; referential integrity enforced; disaster recovery procedures maintained.
- **Database Implementation:** ACID compliance, concurrency control, indexed queries, horizontal scalability, and POPIA/HIPAA audit trails.

2. Database Design

2.1 Business Rules

The following rules are derived from system operations and expressed as formal constraints for database design.

Appointment Booking Rules

- **AB-01:** A patient can book multiple appointments, but each appointment must belong to exactly one patient.
- **AB-02:** Only one staff member (nurse or doctor) may be allocated to each appointment.
- **AB-03:** Every appointment must have a valid date and time.
- **AB-04:** A patient may reschedule or cancel an appointment.

Time Allocation Rules

- **TA-01:** A time slot becomes available when an appointment is cancelled.
- **TA-02:** Every appointment has a single time slot assigned to it.
- **TA-03:** The clinic can accommodate a daily limit of 50 appointments.

Patient Management Rules

- **PM-01:** A student patient must authenticate using their student ID before booking or accessing records.

- **PM-02:** Duplicate patient records are not permitted.
- **PM-03:** Every patient has access to their own personal records.

Staff Management Rules

- **SM-01:** Every staff member must have a defined role.
- **SM-02:** Every doctor must have a licence number and specialisation.
- **SM-03:** Every staff member must have specified working hours.

Walk-in Patient Rules

- **WP-01:** Walk-in patients are allowed without prior booking.
- **WP-02:** Walk-in patients must be assigned an available time slot.

Queue and Check-in Rules

- **QC-01:** A check-in record is required for every appointment.
- **QC-02:** Every consultation must have a recorded start and end time.

System Integrity Rules

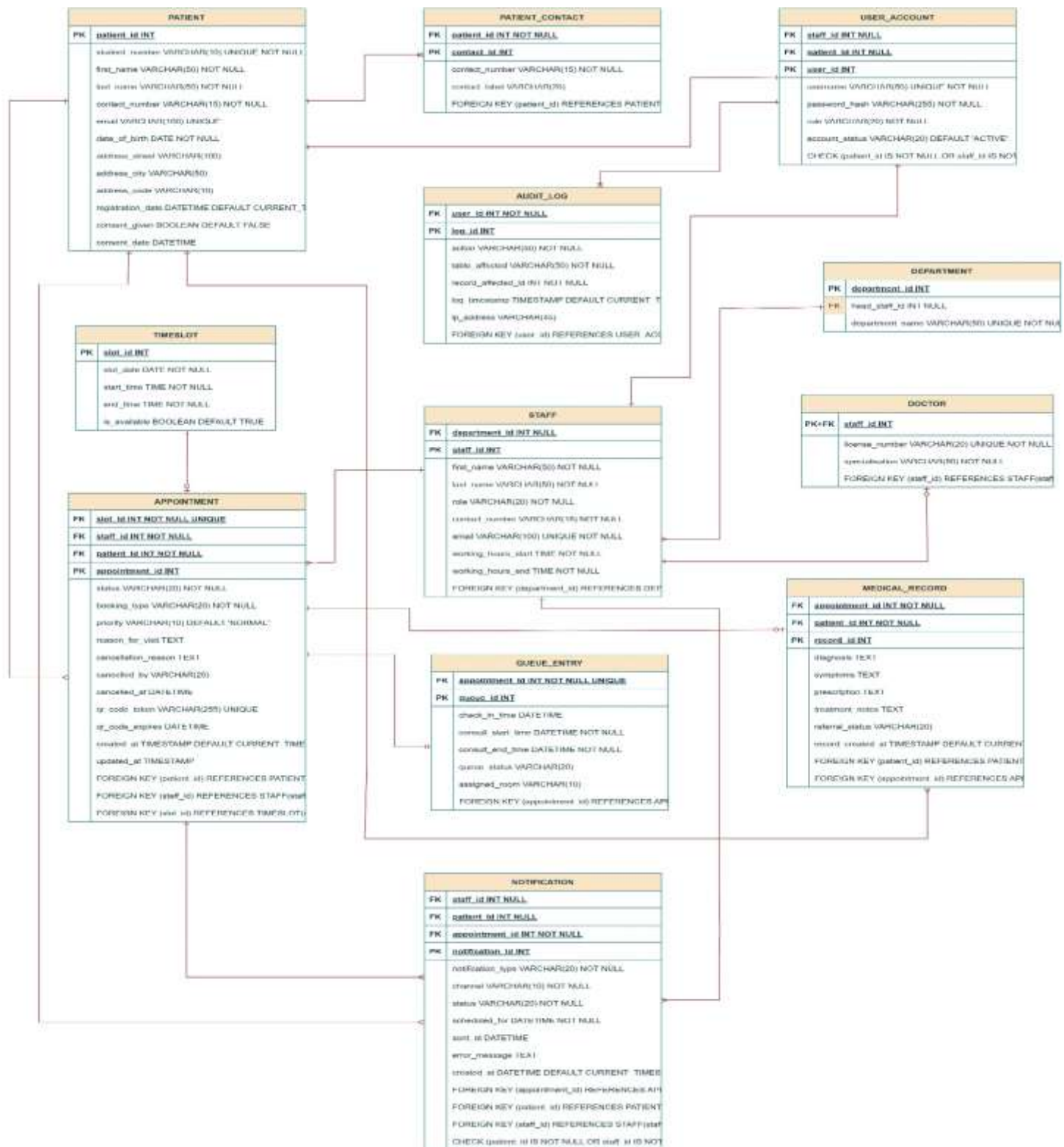
- **SI-01:** All appointments must be validated before confirmation.
- **SI-02:** Medical records must be linked to both a patient and an appointment.

Notification Rule

- **NT-01:** A reminder should be sent 24 hours prior to each scheduled appointment.

2.2 ER Diagram

The ER diagram below depicts all entities, attributes, primary keys, foreign keys, relationships, and their cardinalities. The diagram is fully normalized and clearly indicates optional/mandatory participation, weak entities, composite/bridge entities, composite attributes, multivalued attributes, and derived attributes.



2.3 Logical Data Model

Entities	Cardinality	Description
PATIENT → PATIENT_CONTACT	1:M	A patient may register several contact numbers; each contact belongs to exactly one patient. PATIENT_CONTACT is fully dependent on PATIENT.
PATIENT → USER_ACCOUNT	1:1	A patient may be linked to at most one portal account. Account creation is optional at registration.
USER_ACCOUNT → AUDIT_LOG	1:M	A single account may produce numerous audit log entries. Every log entry requires an actor (user account).
DEPARTMENT → STAFF	1:M	A department may contain several staff members; each staff member belongs to at most one department.
STAFF → DOCTOR	1:0..1	A staff member may optionally hold a doctor role. Every doctor record requires an associated staff record.
STAFF → APPOINTMENT	1:M	A staff member may be scheduled for numerous appointments; each appointment is assigned to exactly one provider.
TIMESLOT → APPOINTMENT	1:1	Each time slot accommodates at most one appointment; every appointment occupies exactly one time slot.
APPOINTMENT → QUEUE_ENTRY	1:1 (Mandatory)	Every appointment generates exactly one queue entry upon patient arrival. QUEUE_ENTRY is a weak entity dependent on APPOINTMENT.
APPOINTMENT → MEDICAL_RECORD	1:0..1	An appointment may result in at most one medical record. Missed appointments generate no documentation.
PATIENT → APPOINTMENT	1:M	A patient may schedule multiple appointments; every appointment requires a patient.
PATIENT → MEDICAL_RECORD	1:M	A patient may accumulate numerous medical records spanning multiple visits.
APPOINTMENT → NOTIFICATION	1:M	An appointment may trigger several notifications throughout its lifecycle.
PATIENT → NOTIFICATION	1:M	A patient may receive numerous system notifications.
STAFF → NOTIFICATION	1:M	A staff member may receive multiple notifications regarding schedule updates and appointments.

2.4 Derived Attributes

Derived attributes are not stored directly in the database but are calculated when needed. This approach reduces data redundancy and eliminates the need for periodic updates.

PATIENT Table – Age

The stored date_of_birth attribute enables dynamic calculation of a patient's current age at query time. Age is computed by subtracting the date of birth from the current date. This eliminates the maintenance burden of updating a static age column annually.

Derivation: Age = CURRENT_DATE - date_of_birth

QUEUE_ENTRY Table – Wait Duration

The duration a patient spends waiting is determined by calculating the interval between `consult_start_time` and `check_in_time`. This derived measurement supports the clinic's objective of maintaining wait times under fifteen minutes without requiring additional stored columns.

Derivation: `wait duration = consult_start_time - check_in_time`

APPOINTMENT Table – No-Show Status

Certain appointment states may be inferred from contextual data rather than stored explicitly. When the scheduled time passes without an associated check-in record in the `QUEUE_ENTRY` table, the appointment can be interpreted as a no-show. This eliminates the need for a dedicated status update.

Derivation: `is_no_show = (appointment_date_time < CURRENT_TIMESTAMP) AND (no matching QUEUE_ENTRY record)`

NOTIFICATION Table – Reminder Status

A reminder is considered "sent" if a `NOTIFICATION` record exists with `notification_type = 'REMINDER'` and `sent_timestamp` is not null. The system checks this 24 hours before each appointment.

Derivation: `reminder_sent = EXISTS (SELECT 1 FROM NOTIFICATION WHERE appointment_id = ? AND type = 'REMINDER')`

2.5 Normalization Description

The database schema has been normalized to Third Normal Form (3NF) to eliminate data redundancy, prevent update anomalies, and ensure data integrity.

First Normal Form (1NF)

Every attribute holds atomic values exclusively.

- Patient address is decomposed into `street`, `city`, and `postal_code` rather than a single concatenated string.
- The `PATIENT_CONTACT` entity stores each phone number in a dedicated row, eliminating multi-valued attributes.
- No repeating groups exist within any table structure.

Second Normal Form (2NF)

All tables use single-column surrogate primary keys, eliminating partial dependencies.

- Each table has a dedicated primary key (`patient_id`, `appointment_id`, `staff_id`, etc.).
- Where candidate keys exist (e.g., `student_number`), all non-key attributes remain fully dependent on the primary key alone.
- No composite keys are used, so partial dependencies are impossible.

Third Normal Form (3NF)

The schema contains no transitive dependencies.

- The `DOCTOR` subtype table removes `license_number` and `specialisation` from `STAFF`, as these apply only to doctors. This prevents NULL values for non-doctors and avoids update anomalies.
- The `APPOINTMENT` table references staff via `staff_id` rather than duplicating doctor names or licence numbers. Changes to staff information occur in one place only.
- `MEDICAL_RECORD` links to `PATIENT` and `APPOINTMENT` via foreign keys instead of storing redundant patient names or appointment dates.

2.6 Logical Data Model

Each entity in the ER diagram maps to a relation below. Primary keys are underlined. Foreign keys are indicated with (FK).

PATIENT

patient_id (PK), student_number, first_name, last_name, date_of_birth, email_address, residential_address_street, residential_address_city, residential_address_postal, next_of_kin_name, next_of_kin_phone, medical_aid_policy_number, medical_aid_scheme, registration_date

PATIENT_CONTACT

contact_id (PK), patient_id (FK → PATIENT), contact_number, contact_type

USER_ACCOUNT

user_id (PK), patient_id (FK → PATIENT, nullable), staff_id (FK → STAFF, nullable), username, password_hash, role, account_status

DEPARTMENT

department_id (PK), department_name, head_staff_id (FK → STAFF, nullable)

STAFF

staff_id (PK), department_id (FK → DEPARTMENT, nullable), first_name, last_name, staff_role, contact_number, email, working_hours, consultation_room

DOCTOR

doctor_id (PK, FK → STAFF), license_number, specialisation

TIMESLOT

slot_id (PK), slot_date, slot_start_time, slot_end_time, is_available

APPOINTMENT

appointment_id (PK), patient_id (FK → PATIENT), staff_id (FK → STAFF), slot_id (FK → TIMESLOT, unique), reason_for_visit, appointment_status, appointment_type, priority_level, created_timestamp, last_modified_timestamp

QUEUE_ENTRY

queue_id (PK), appointment_id (FK → APPOINTMENT, unique), check_in_time, consult_start_time, consult_end_time, assigned_room, queue_status

MEDICAL_RECORD

record_id (PK), patient_id (FK → PATIENT), appointment_id (FK → APPOINTMENT, unique), diagnosis, symptoms, prescription, treatment_notes, referral_status, record_creation_timestamp

NOTIFICATION

notification_id (PK), appointment_id (FK → APPOINTMENT), patient_id (FK → PATIENT, nullable), staff_id (FK → STAFF, nullable), notification_type, message, sent_timestamp, delivery_status

AUDIT_LOG

log_id (PK), user_id (FK → USER_ACCOUNT), action, table_affected, record_id, action_timestamp, ip_address

2.7 Summary of Entities

Entity	Primary Key	Foreign Key(s)
PATIENT	patient_id	-
PATIENT_CONTACT	contact_id	patient_id
USER_ACCOUNT	user_id	patient_id, staff_id
DEPARTMENT	department_id	head_staff_id
STAFF	staff_id	department_id
DOCTOR	doctor_id	doctor_id (to STAFF)
TIMESLOT	slot_id	-
APPOINTMENT	appointment_id	patient_id, staff_id, slot_id
QUEUE_ENTRY	queue_id	appointment_id
MEDICAL_RECORD	record_id	patient_id, appointment_id
NOTIFICATION	notification_id	appointment_id, patient_id, staff_id
AUDIT_LOG	log_id	user_id

3. Physical Design

3.1 Technology Stack Summary

Layer	Technology	Purpose
SQL Database (Rubric)	Oracle XE 21c	SQL queries, views, indexes, Oracle SQL Developer
Production Database	PostgreSQL 15	Backend CRUD, live system data on Render.com
Backend Framework	FastAPI (Python)	REST API, JWT authentication
Frontend Framework	React 18 + Vite	User interface
Styling	Tailwind CSS	UI components and responsive design
Authentication	JWT (JSON Web Tokens)	Role-based access control
Testing	pytest	Unit tests for backend
Deployment	Render.com	Hosting backend + frontend + PostgreSQL
CI/CD	GitHub Actions	Automated testing on pull requests
Version Control	Git + GitHub	Code repository

3.2 Database Objects

3.2.1 Tables and Data Types(Oracle Syntax)

All tables are implemented in **Oracle XE 21c** using Oracle-compatible data types. The same schema structure is mirrored in **PostgreSQL 15** for the live backend system.

PATIENT

Column	Data Type	Constraints
patient_id	NUMBER GENERATED BY DEFAULT AS IDENTITY	PRIMARY KEY
student_number	VARCHAR2(20)	NOT NULL, UNIQUE
first_name	VARCHAR2(50)	NOT NULL
last_name	VARCHAR2(50)	NOT NULL
date_of_birth	DATE	NOT NULL
email_address	VARCHAR2(100)	NOT NULL, UNIQUE
residential_address_street	VARCHAR2(100)	NOT NULL
residential_address_city	VARCHAR2(50)	NOT NULL
residential_address_postal	VARCHAR2(10)	NOT NULL
next_of_kin_name	VARCHAR2(100)	NOT NULL
next_of_kin_phone	VARCHAR2(15)	NOT NULL
medical_aid_policy_number	VARCHAR2(50)	NULL
medical_aid_scheme	VARCHAR2(50)	NULL
registration_date	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP

PATIENT_CONTACT

Column	Data Type	Constraints
contact_id	NUMBER GENERATED BY DEFAULT AS IDENTITY	PRIMARY KEY
patient_id	NUMBER	NOT NULL, FOREIGN KEY (PATIENT)
contact_number	VARCHAR2(15)	NOT NULL
contact_type	VARCHAR2(20)	NOT NULL

USER_ACCOUNT

Column	Data Type	Constraints
user_id	NUMBER GENERATED BY DEFAULT AS IDENTITY	PRIMARY KEY
patient_id	NUMBER	NULL, FOREIGN KEY (PATIENT)
staff_id	NUMBER	NULL, FOREIGN KEY (STAFF)
username	VARCHAR2(50)	NOT NULL, UNIQUE
password_hash	VARCHAR2(255)	NOT NULL
role	VARCHAR2(20)	NOT NULL
account_status	VARCHAR2(20)	NOT NULL, DEFAULT 'ACTIVE'

DEPARTMENT

Column	Data Type	Constraints
department_id	NUMBER GENERATED BY DEFAULT AS IDENTITY	PRIMARY KEY
department_name	VARCHAR2(50)	NOT NULL, UNIQUE
head_staff_id	NUMBER	NULL, FOREIGN KEY (STAFF)

STAFF

Column	Data Type	Constraints
staff_id	NUMBER GENERATED BY DEFAULT AS IDENTITY	PRIMARY KEY
department_id	NUMBER	NULL, FOREIGN KEY (DEPARTMENT)
first_name	VARCHAR2(50)	NOT NULL
last_name	VARCHAR2(50)	NOT NULL
staff_role	VARCHAR2(50)	NOT NULL
contact_number	VARCHAR2(15)	NOT NULL
email	VARCHAR2(100)	NOT NULL, UNIQUE
working_hours	VARCHAR2(100)	NOT NULL
consultation_room	VARCHAR2(10)	NULL

DOCTOR

Column	Data Type	Constraints
doctor_id	NUMBER	PRIMARY KEY, FOREIGN KEY (STAFF)
license_number	VARCHAR2(50)	NOT NULL, UNIQUE
specialisation	VARCHAR2(50)	NOT NULL

TIMESLOT

Column	Data Type	Constraints
slot_id	NUMBER GENERATED BY DEFAULT AS IDENTITY	PRIMARY KEY
slot_date	DATE	NOT NULL
slot_start_time	TIMESTAMP	NOT NULL
slot_end_time	TIMESTAMP	NOT NULL
is_available	NUMBER(1)	NOT NULL, DEFAULT 1

APPOINTMENT

Column	Data Type	Constraints
appointment_id	NUMBER GENERATED BY DEFAULT AS IDENTITY	PRIMARY KEY
patient_id	NUMBER	NOT NULL, FOREIGN KEY (PATIENT)
staff_id	NUMBER	NOT NULL, FOREIGN KEY (STAFF)
slot_id	NUMBER	NOT NULL, FOREIGN KEY (TIMESLOT), UNIQUE
reason_for_visit	VARCHAR2(500)	NOT NULL
appointment_status	VARCHAR2(20)	NOT NULL
appointment_type	VARCHAR2(20)	NOT NULL
priority_level	VARCHAR2(10)	NOT NULL, DEFAULT 'NORMAL'
created_timestamp	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
last_modified_timestamp	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP

QUEUE_ENTRY

Column	Data Type	Constraints
queue_id	NUMBER GENERATED BY DEFAULT AS IDENTITY	PRIMARY KEY
appointment_id	NUMBER	NOT NULL, FOREIGN KEY (APPOINTMENT), UNIQUE
check_in_time	TIMESTAMP	NOT NULL
consult_start_time	TIMESTAMP	NULL
consult_end_time	TIMESTAMP	NULL
assigned_room	VARCHAR2(10)	NULL
queue_status	VARCHAR2(20)	NOT NULL

MEDICAL_RECORD

Column	Data Type	Constraints
record_id	NUMBER GENERATED BY DEFAULT AS IDENTITY	PRIMARY KEY
patient_id	NUMBER	NOT NULL, FOREIGN KEY (PATIENT)
appointment_id	NUMBER	NOT NULL, FOREIGN KEY (APPOINTMENT), UNIQUE
diagnosis	VARCHAR2(500)	NULL
symptoms	VARCHAR2(500)	NOT NULL
prescription	VARCHAR2(500)	NULL
treatment_notes	VARCHAR2(1000)	NULL
referral_status	VARCHAR2(50)	NULL
record_creation_timestamp	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP

NOTIFICATION

Column	Data Type	Constraints
notification_id	NUMBER GENERATED BY DEFAULT AS IDENTITY	PRIMARY KEY
appointment_id	NUMBER	NOT NULL, FOREIGN KEY (APPOINTMENT)
patient_id	NUMBER	NULL, FOREIGN KEY (PATIENT)
staff_id	NUMBER	NULL, FOREIGN KEY (STAFF)

notification_type	VARCHAR2(20)	NOT NULL
message	VARCHAR2(1000)	NOT NULL
sent_timestamp	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
delivery_status	VARCHAR2(20)	NOT NULL

AUDIT_LOG

Column	Data Type	Constraints
log_id	NUMBER GENERATED BY DEFAULT AS IDENTITY	PRIMARY KEY
user_id	NUMBER	NOT NULL, FOREIGN KEY (USER_ACCOUNT)
action	VARCHAR2(10)	NOT NULL
table_affected	VARCHAR2(50)	NOT NULL
record_id	NUMBER	NOT NULL
action_timestamp	TIMESTAMP	NOT NULL, DEFAULT CURRENT_TIMESTAMP
ip_address	VARCHAR2(45)	NULL

3.2.2 Constraints Summary

Constraint Type	Applied To	Purpose
PRIMARY KEY	All tables (id columns)	Uniquely identify each row
FOREIGN KEY	patient_id, staff_id, slot_id, etc.	Enforce referential integrity
NOT NULL	Key fields	Prevent null values in required fields
UNIQUE	student_number, email, username, license_number, slot_id	Prevent duplicate values
DEFAULT	registration_date, is_available, priority_level, timestamps	Set default values automatically
CHECK	role, account_status, appointment_status, queue_status	Restrict values to valid options

3.2.3 Indexes

Index Name	Table	Column(s)	Type	Purpose
idx_appointment_patient	APPOINTMENT	patient_id	B-Tree	Fast lookup of patient appointments
idx_appointment_staff	APPOINTMENT	staff_id	B-Tree	Fast lookup of staff schedules
idx_appointment_slot	APPOINTMENT	slot_id	UNIQUE	Enforce one appointment per slot
idx_appointment_date	APPOINTMENT	created_timestamp	B-Tree	Date range queries
idx_patient_student	PATIENT	student_number	UNIQUE	Fast login/authentication
idx_user_username	USER_ACCOUNT	username	UNIQUE	Fast login lookups
idx_user_role	USER_ACCOUNT	role	B-Tree	Role-based access filtering
idx_queue_status	QUEUE_ENTRY	queue_status	B-Tree	Waiting room queries
idx_notification_appointment	NOTIFICATION	appointment_id	B-Tree	Appointment reminder lookups
idx_audit_user	AUDIT_LOG	user_id	B-Tree	Audit trail by user
idx_audit_timestamp	AUDIT_LOG	action_timestamp	B-Tree	Audit date range queries
idx_timeslot_date	TIMESLOT	slot_date	B-Tree	Daily slot availability
idx_medical_patient	MEDICAL_RECORD	patient_id	B-Tree	Patient medical history lookup
idx_doctor_specialisation	DOCTOR	specialisation	B-Tree	Filter doctors by specialisation
idx_appointment_status	APPOINTMENT	appointment_status	B-Tree	Filter by appointment status

3.2.4 Four Views (Oracle Syntax) –

View 1: Active Appointments View -Sql

```
CREATE VIEW VW_PATIENT_APPOINTMENTS AS
SELECT
  p.patient_id,
  p.first_name || ' ' || p.last_name AS patient_name,
  a.appointment_id,
  t.slot_date,
  t.slot_start_time,
  t.slot_end_time,
  s.first_name || ' ' || s.last_name AS doctor_name,
  a.appointment_status,
  a.priority_level
FROM PATIENT p
JOIN APPOINTMENT a ON p.patient_id = a.patient_id
JOIN TIMESLOT t ON a.slot_id = t.slot_id
JOIN STAFF s ON a.staff_id = s.staff_id
ORDER BY t.slot_date DESC, t.slot_start_time ASC;
```

Purpose: Provides a complete view of all patient appointments with patient names, doctor names, time slots, and statuses.

View 2: VW_DAILY_QUEUE_BOARD -Sql

```
CREATE VIEW VW_DAILY_QUEUE_BOARD AS
SELECT
  q.queue_id,
  p.first_name || ' ' || p.last_name AS patient_name,
  q.check_in_time,
  q.queue_status,
  a.priority_level,
  ROUND((SYSDATE - q.check_in_time) * 24 * 60, 0) AS wait_minutes,
  a.appointment_id
FROM QUEUE_ENTRY q
JOIN APPOINTMENT a ON q.appointment_id = a.appointment_id
JOIN PATIENT p ON a.patient_id = p.patient_id
WHERE q.queue_status = 'WAITING'
ORDER BY a.priority_level DESC, q.check_in_time ASC;
```

Purpose: Displays current waiting patients sorted by priority (urgent first) and check-in time. Calculates wait minutes dynamically using Oracle date arithmetic.

View 3: VW_DOCTOR_SCHEDULE -Sql

```
CREATE VIEW VW_DOCTOR_SCHEDULE AS
SELECT
  s.staff_id,
  s.first_name || ' ' || s.last_name AS doctor_name,
  d.specialisation,
  t.slot_date,
  t.slot_start_time,
  t.slot_end_time,
  a.appointment_id,
  p.first_name || ' ' || p.last_name AS patient_name,
  a.appointment_status
FROM STAFF s
JOIN DOCTOR d ON s.staff_id = d.doctor_id
LEFT JOIN APPOINTMENT a ON s.staff_id = a.staff_id
LEFT JOIN TIMESLOT t ON a.slot_id = t.slot_id
LEFT JOIN PATIENT p ON a.patient_id = p.patient_id
WHERE s.staff_role = 'DOCTOR'
ORDER BY t.slot_date, t.slot_start_time;
```

Purpose: Shows each doctor's schedule including appointments, patient names, and time slots. Left joins ensure doctors with no appointments still appear.

View 4: VW_AUDIT_TRAIL_SUMMARY -Sql

```
CREATE VIEW VW_AUDIT_TRAIL_SUMMARY AS
SELECT
    al.log_id,
    ua.username,
    ua.role AS user_role,
    al.action,
    al.table_affected,
    al.record_id,
    al.action_timestamp,
    al.ip_address
FROM AUDIT_LOG al
JOIN USER_ACCOUNT ua ON al.user_id = ua.user_id
ORDER BY al.action_timestamp DESC;
```

Purpose: Provides a complete audit trail for POPIA compliance, showing who performed what action, on which table, and when.

3.3 Data Populated

Seed Data Summary

Table	Rows Populated	Description
PATIENT	10	Sample patients with student numbers and contact details
PATIENT_CONTACT	15	Multiple contact numbers per patient
DEPARTMENT	5	Medical, Nursing, Pharmacy, Laboratory, IT
STAFF	8	Doctors, nurses, admin staff
DOCTOR	3	Doctors with licence numbers and specialisations
TIMESLOT	50	Daily slots across 5 days (10 slots per day)
APPOINTMENT	15	Mixed statuses (scheduled, completed, cancelled)
QUEUE_ENTRY	8	Check-in records for completed appointments
MEDICAL_RECORD	6	Medical histories for completed consultations
USER_ACCOUNT	12	Accounts for patients and staff
NOTIFICATION	10	Reminder and confirmation notifications
AUDIT_LOG	20	System activity logs

Sample INSERT Statements

PATIENT:

sql

```
INSERT INTO PATIENT (student_number, first_name, last_name, date_of_birth, email_address,
residential_address_street, residential_address_city, residential_address_postal, next_of_kin_name,
next_of_kin_phone) VALUES
```

```
('202400001', 'John', 'Doe', '2000-05-15', 'john.doe@nwu.ac.za', '123 University Road', 'Potchefstroom', '2520',
'Jane Doe', '0821234567');
```

STAFF:

```
sql
INSERT INTO STAFF (department_id, first_name, last_name, staff_role, contact_number, email,
working_hours, consultation_room) VALUES
(1, 'Sarah', 'Johnson', 'Doctor', '0825551234', 'sarah.johnson@nwu.ac.za', 'Mon-Fri 08:00-16:00', '101');
```

APPOINTMENT:

```
sql
INSERT INTO APPOINTMENT (patient_id, staff_id, slot_id, reason_for_visit, appointment_status,
appointment_type, priority_level) VALUES
(1, 1, 1, 'Fever and cough', 'SCHEDULED', 'SICK', 'URGENT');
```

4. SQL Queries**Query Categories**

All queries are available in sql/04_queries/05_queries.sql and were executed in Oracle SQL Developer.

Q1 – Business Queries

Query	Purpose	Expected Outcome
Q1.1	List all patients registered in the last 30 days	Returns patient names, student numbers, and registration dates
Q1.3	Show all available time slots for today	Returns slot times where is_available = 1

Q2 – Row/Column Limits

Query	Purpose	Expected Outcome
Q2.1	Fetch first 5 patients	Returns 5 patient records ordered by registration date

Q3 – Sorting

Query	Purpose	Expected Outcome
Q3.1	Sort appointments by date DESC then status ASC	Shows newest appointments first, grouped by status

Q4 – Operators

Query	Purpose	Expected Outcome
Q4.1	Find patients with email ending in @nwu.ac.za	Returns all NWU student email addresses

Q5 – Character Functions + Substitution Variable

Query	Purpose	Expected Outcome
Q5.6	Substitution variable (&student_number)	Prompts user for student number and returns matching patient

Highlight: Q5.6 opens a pop-up dialog in SQL Developer asking for a student number (e.g., 48277444). This demonstrates the &variable feature required in the rubric.

Q6 – Date Functions

Query	Purpose	Expected Outcome
Q7.1	SYSDATE – current system date	Shows today's date from Oracle

Q7 – Aggregate Functions

Query	Purpose	Expected Outcome
Q8.2	AVG (wait_time) – average wait time	Returns mean patient wait time

Q8 – GROUP BY + HAVING

Query	Purpose	Expected Outcome
Q9.2	GROUP BY appointment_status	Shows counts of SCHEDULED, COMPLETED, CANCELLED, NO_SHOW

Q9 – Joins

Query	Purpose	Expected Outcome
Q10.4	5-table JOIN: APPOINTMENT + PATIENT + TIMESLOT + STAFF + QUEUE_ENTRY	Returns complete clinical record including check-in and queue status

Highlight: Q10.4 joins 5 tables in a single statement, directly reflecting the ER diagram relationships working in practice.

Q10 – Window Function

Query	Purpose	Expected Outcome
Q11.6	RANK () window function – rank doctors by appointment count	Returns doctors ranked from busiest to least busy

Highlight: Q11.6 uses Oracle's RANK () analytical window function to rank doctors by appointment workload. This demonstrates Oracle-specific expertise beyond standard SQL.

Sample Query (Q10.4 - 5-table JOIN)

```
SELECT
  p.first_name || ' ' || p.last_name AS patient_name,
  s.first_name || ' ' || s.last_name AS doctor_name,
  t.slot_date,
  t.slot_start_time,
  q.check_in_time,
  q.queue_status,
  a.appointment_status
FROM APPOINTMENT a
JOIN PATIENT p ON a.patient_id = p.patient_id
JOIN TIMESLOT t ON a.slot_id = t.slot_id
JOIN STAFF s ON a.staff_id = s.staff_id
LEFT JOIN QUEUE_ENTRY q ON a.appointment_id = q.appointment_id;
```

Summary of Query Categories

Category	Number of Queries	Key Features Demonstrated
Business Queries	2	Real-world reporting needs
Row Limits	1	FETCH FIRST
Sorting	1	Multi-column ASC/DESC
Operators	1	LIKE
Character + Substitution	1	&variable pop-up dialog
Date Functions	1	SYSDATE
Aggregate Functions	1	AVG
GROUP BY	1	Status distribution
Joins	1	5-table JOIN
Window Function	1	RANK ()
Total	11	Full rubric coverag

5. Extra Functionalities

Extra functionalities are demonstrated in the video. These include JWT role-based authentication (Patient, Doctor, Nurse, Admin), QR code check-in system, real-time queue management board, automated email notifications, POPIA-compliant audit logging, role-based dashboards, daily capacity management (50 patients per day), and four Oracle reporting views.

6. Demonstration Video

A demonstration video has been submitted as part of the Phase 3 deliverables. The video shows the Oracle SQL Developer setup, all 49 SQL queries running successfully, and the live system including patient registration, appointment booking, QR check-in, queue management, role-based access control, and the audit log for POPIA compliance. The video is available on Dropbox and has been uploaded to eFundi as per the submission requirements.

Conclusion

Summary of Work Completed

This report documented the complete database project undertaken by Group 19 for the Ubuntu Campus Clinic Appointment System, developed as part of the CMPG 311 module at North-West University.

Phase 1 – Database Initial Study established a thorough understanding of the Ubuntu Campus Clinic's operations. We identified six core problems with the manual appointment system: delays in accessing healthcare, limited operating hours, long waiting periods, paper-based scheduling, limited capacity, and unreliable service delivery. We also defined comprehensive database system specifications including objectives, required information, scope, and boundaries.

Phase 2 – Database Design translated these requirements into a normalized conceptual design. We produced formal business rules, a complete Entity-Relationship (ER) diagram with 12 entities and all cardinalities, derived attributes, and a fully normalized logical data model achieving Third Normal Form (3NF). The design addressed all identified operational problems through appropriate entity relationships and constraints.

Phase 3 – Physical Design implemented the database using Oracle XE 21c for SQL queries and PostgreSQL 15 for the live backend system. The backend was built with FastAPI (Python) and the frontend used React 18 with Tailwind CSS. The system includes 12 normalized tables, 4 reporting views, 15 performance indexes, and SQL queries covering all rubric categories. Extra functionalities include JWT role-based authentication, QR code check-in, automated email notifications, and POPIA-compliant audit logging.

Reflections on Project Challenges and Solutions

Challenge 1 – Defining appropriate cardinalities: Determining the correct cardinality for the STAFF to DOCTOR relationship was initially difficult. We resolved this by analysing business rules carefully, leading to the correct 1:0..1 subtype relationship.

Challenge 2 – Managing optional vs mandatory participation: Several entities had optional participation. We implemented nullable foreign keys with CHECK constraints to maintain flexibility while preserving data integrity.

Challenge 3 – Handling walk-in patients without prior appointments: We added appointment_type = 'WALK_IN' to the APPOINTMENT table and allowed the system to assign available TIMESLOT entries in real-time.

Challenge 4 – Real-time queue management: We created a waiting room view that dynamically calculates wait times using Oracle date arithmetic and sorts by priority level.

Challenge 7 – Team coordination across frontend and backend: With nine team members split between frontend, backend, and integration, we used GitHub pull requests requiring approval and CI pipelines running pytest on every PR to catch integration issues early.

Final Outcome

Everything worked. The completed Clinic Appointment System successfully meets all project objectives:

- Same-day bookings for urgent cases
- 24/7 online appointment access
- Real-time queue management with wait times under 15 minutes
- Capacity management with a daily limit of 50 patients
- POPIA-compliant audit trails
- Role-based access control for patients, doctors, nurses, and administrators

All SQL queries run successfully in Oracle SQL Developer. All 12 tables are populated with seed data. The 4 views return correct results. The 15 indexes are in place. The live system is deployed on [Render.com](https://render.com) with FastAPI backend and React frontend communicating seamlessly with PostgreSQL. The demonstration video has been submitted via Dropbox and eFundi.

References

Academic & Books

- Banks, A. and Porcello, E. (2024) *Learning React: Modern Patterns for Developing React Apps*. 3rd edn. Sebastopol: O'Reilly Media.
- Brown, K.C. and Vessey, I. (2003) *Managing the Next Wave of Enterprise Systems: Leveraging Lessons from ERP*. Boston, MA: MIT Press.
- Connolly, T. and Begg, C. (2015) *Database Systems: A Practical Approach to Design, Implementation, and Management*. 6th edn. Harlow: Pearson Education.
- Elmasri, R. and Navathe, S.B. (2016) *Fundamentals of Database Systems*. 7th edn. Boston: Pearson.
- Fielding, R.T. (2000) *Architectural Styles and the Design of Network-based Software Architectures*. Doctoral dissertation. University of California, Irvine.
- Percival, H. and Gregory, B. (2024) *Test-Driven Development with Python*. 2nd edn. Shelter Island: Manning Publications.
- Wieruch, R. (2023) *The Road to React: Your Journey to Master React*. 2nd edn. Amsterdam: The Road to React Publishing.

Official Documentation

- Auth0 (2025) *Introduction to JSON Web Tokens*. Available at: <https://jwt.io/introduction> (Accessed: 17 May 2026).
- Axios Documentation (2025) *Axios: Promise based HTTP client for the browser and Node.js*. Available at: <https://axios-http.com> (Accessed: 9 May 2026, 10 May 2026, 16 May 2026, 17 May 2026).
- Flask Documentation (2024) *Flask Documentation*. Pallets Projects. Available at: <https://flask.palletsprojects.com> (Accessed: 15 May 2026, 17 May 2026).
- Git Documentation (2026) *Git documentation*. Available at: <https://git-scm.com/doc> (Accessed: 14 May 2026, 17 May 2026).
- GitHub Inc. (2025) *GitHub Documentation*. Available at: <https://docs.github.com> (Accessed: 17 May 2026).
- Gupta, L. (2025) *REST API Tutorial*. Available at: <https://restfulapi.net> (Accessed: 17 May 2026).
- Microsoft (2025) *Visual Studio Code Documentation*. Available at: <https://code.visualstudio.com/docs> (Accessed: 17 May 2026).
- Microsoft Learn (2025) *Software Development Lifecycle (SDLC) Overview*. Available at: <https://learn.microsoft.com/en-us/devops/plan/what-is-sdlc> (Accessed: 17 May 2026).
- Oracle Corporation (2021) *Oracle Database 21c Express Edition*. Available at: <https://www.oracle.com/database/technologies/xe-downloads.html> (Accessed: 15 May 2026).

Oracle Corporation (2025) *Oracle Application Development Framework Documentation*. Available at: <https://www.oracle.com/application-development/technologies/adf.html> (Accessed: 17 May 2026).

Oracle Corporation (2025) *Oracle Database Documentation*. Available at: <https://docs.oracle.com/en/database/oracle/oracle-database/> (Accessed: 16 May 2026, 17 May 2026).

Oracle (2025) *python-oracledb Documentation*. Available at: <https://python-oracledb.readthedocs.io> (Accessed: 15 May 2026, 17 May 2026).

Penpot (2023) *Tutorial for beginners. Penpot Libraries & Templates*. Available at: <https://penpot.app/penpothub/libraries-templates/tutorial-for-beginners> (Accessed: 9 May 2026).

Penpot (2024) *Tutorial: Create and share components in Penpot*. Available at: <https://penpot.app/blog/tutorial-create-and-share-components-in-penpot/> (Accessed: 9 May 2026).

PyJWT (2024) *PyJWT Documentation*. Available at: <https://pyjwt.readthedocs.io> (Accessed: 15 May 2026).

Pytest Documentation (2026) *Pytest documentation*. Available at: <https://docs.pytest.org/en/stable/> (Accessed: 10 May 2026).

Python Documentation (2026) *Python 3 documentation*. Available at: <https://docs.python.org/3/> (Accessed: 10 May 2026, 17 May 2026).

Python Software Foundation (2025) *Python Documentation*. Available at: <https://docs.python.org> (Accessed: 17 May 2026).

QRCode.react Documentation (2025) *qrcode.react: QR code generator for React*. Available at: <https://github.com/zpao/qrcode.react> (Accessed: 16 May 2026, 17 May 2026).

React Documentation (2025) *React: The Library for Web and Native User Interfaces*. Available at: <https://react.dev> (Accessed: 9 May 2026, 10 May 2026, 16 May 2026, 17 May 2026).

React Big Calendar Documentation (2025) *react-big-calendar: Calendar component for React*. Available at: <https://github.com/jquense/react-big-calendar> (Accessed: 16 May 2026, 17 May 2026).

React Router Documentation (2025) *React Router: Declarative Routing for React*. Available at: <https://reactrouter.com> (Accessed: 16 May 2026, 17 May 2026).

Tailwind CSS Documentation (2025) *Tailwind CSS: Utility-First CSS Framework*. Available at: <https://tailwindcss.com> (Accessed: 16 May 2026, 17 May 2026).

Werkzeug Documentation (2024) *Werkzeug Documentation*. Pallets Projects. Available at: <https://werkzeug.palletsprojects.com> (Accessed: 15 May 2026).

Online Tutorials & Videos

Ahmed (2026) *Penpot Tutorial: Master Flex Layout & Responsive Design*. 25 January. Available at: https://youtu.be/Df8FmRddEeo?si=DMkdDIAYFLVX_acl (Accessed: 8 May 2026).

Bolt (2025) *Bolt AI builder: Websites, apps & prototypes*. Available at: <https://bolt.new> (Accessed: 9 May 2026).

Penpot (2024) *Wireframing | Penpot Hands-On Demo*. 28 August. Available at: https://www.youtube.com/live/BVJaiwaN5aA?si=Z_1QXjfRl6VoXMCH (Accessed: 9 May 2026).

Penpot (2026) *Get started with Penpot in under 5 minutes*. 16 March. Available at: <https://youtu.be/vAkb4QQmbfk?si=Gsku-1bWljDzXb8> (Accessed: 9 May 2026).

GitHub & Devpost References

Anis08 (2025) *Cabinet Médical - Medical Practice Management Application*. Available at: <https://github.com/Anis08/cabinet-medical-app> (Accessed: 9 May 2026, 10 May 2026).

md-ajim (2026) *Python exercises and practice projects*. Available at: <https://github.com/md-ajim> (Accessed: 16 May 2026, 17 May 2026).

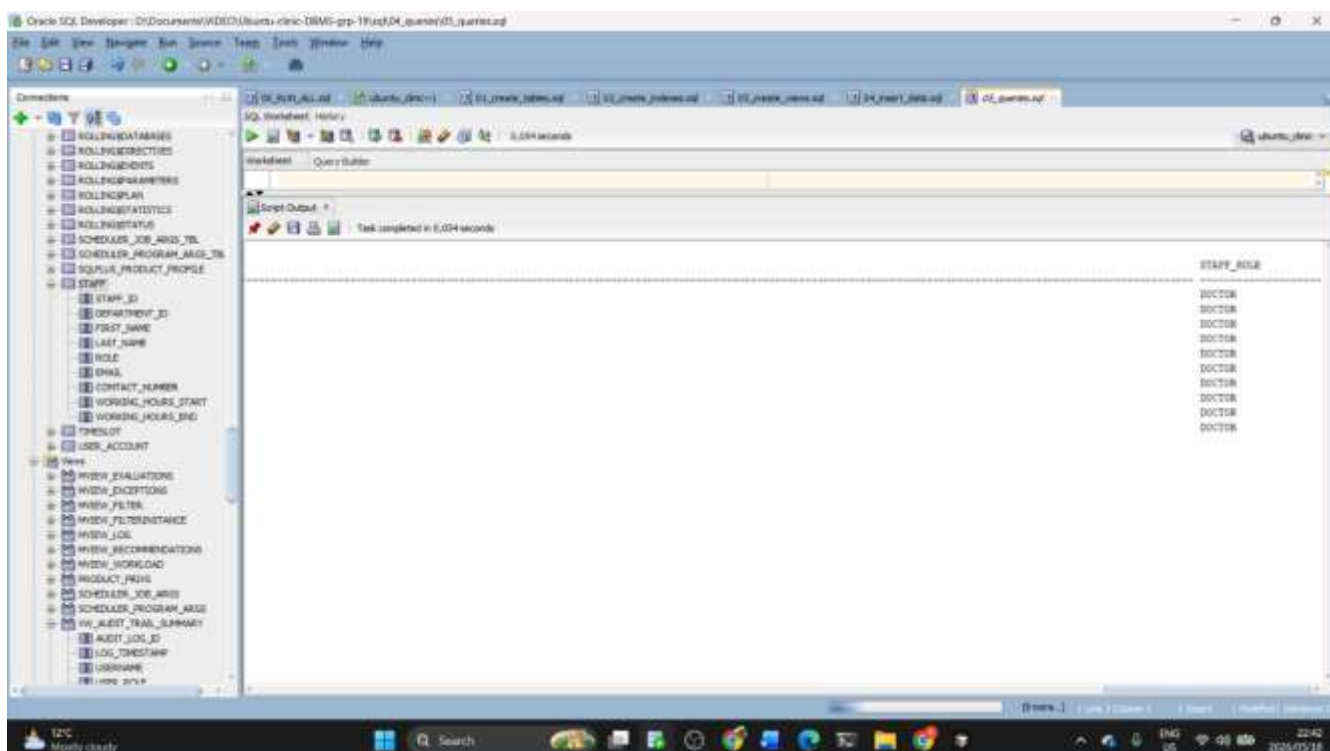
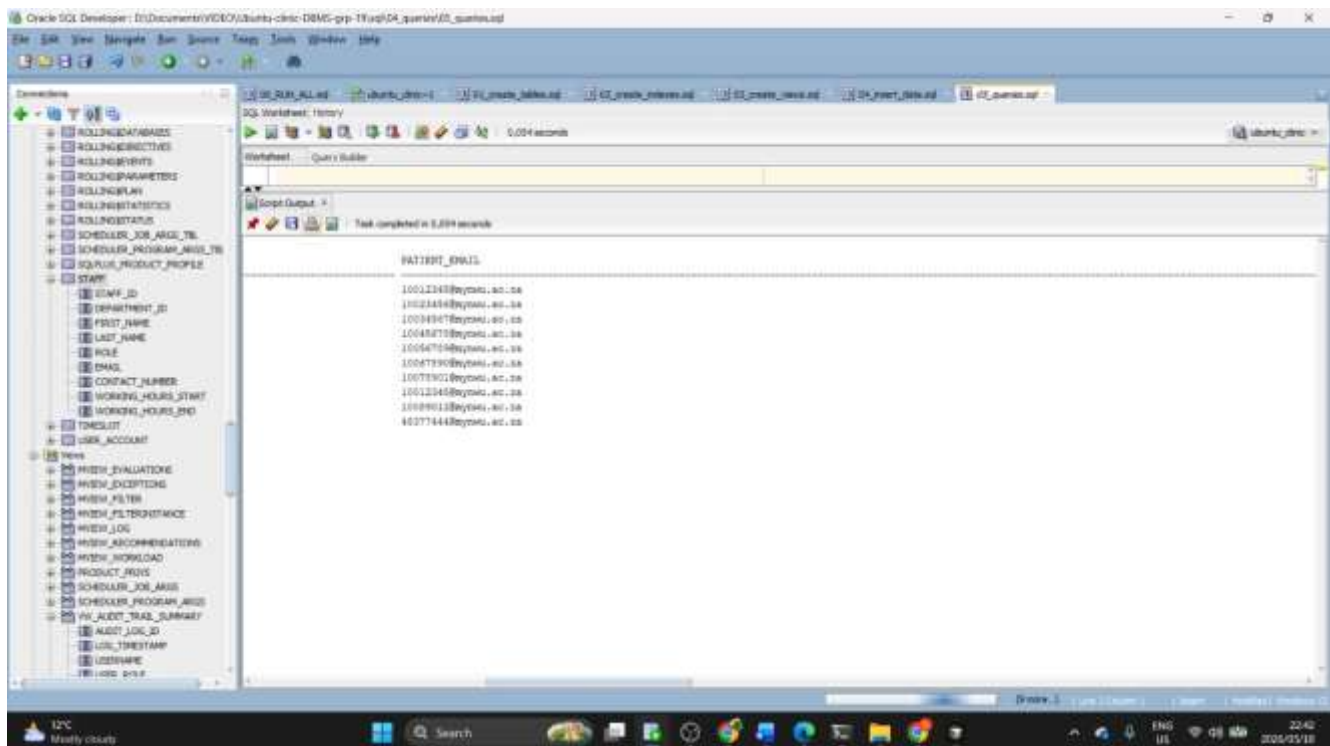
RayeCure (2025) *RayeCure Queue Management System*. Available at: <https://devpost.com/software/rayecure-queue-management-system> (Accessed: 16 May 2026, 17 May 2026).

supravat011 (2025) *EHR NOW - Hospital Management System*. Available at: <https://github.com/supravat011/Hospital-Management> (Accessed: 9 May 2026, 10 May 2026).

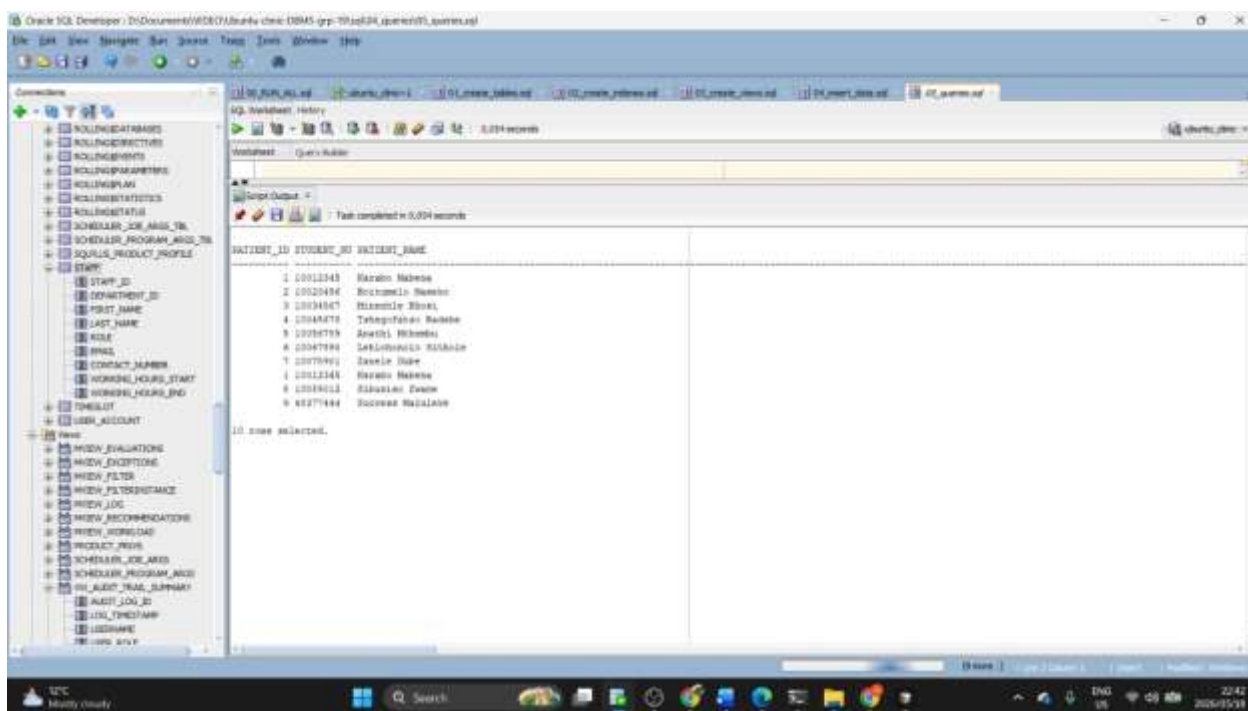
Appendices

Appendix A: Oracle SQL Developer Setup

Screenshots showing the Oracle SQL Developer environment, connection to the database, and the SQL file structure including 00_RUN_ALL.sql, 01_create_tables.sql, 02_create_indexes.sql, 03_create_views.sql, 04_insert_data.sql, and 05_queries.sql.

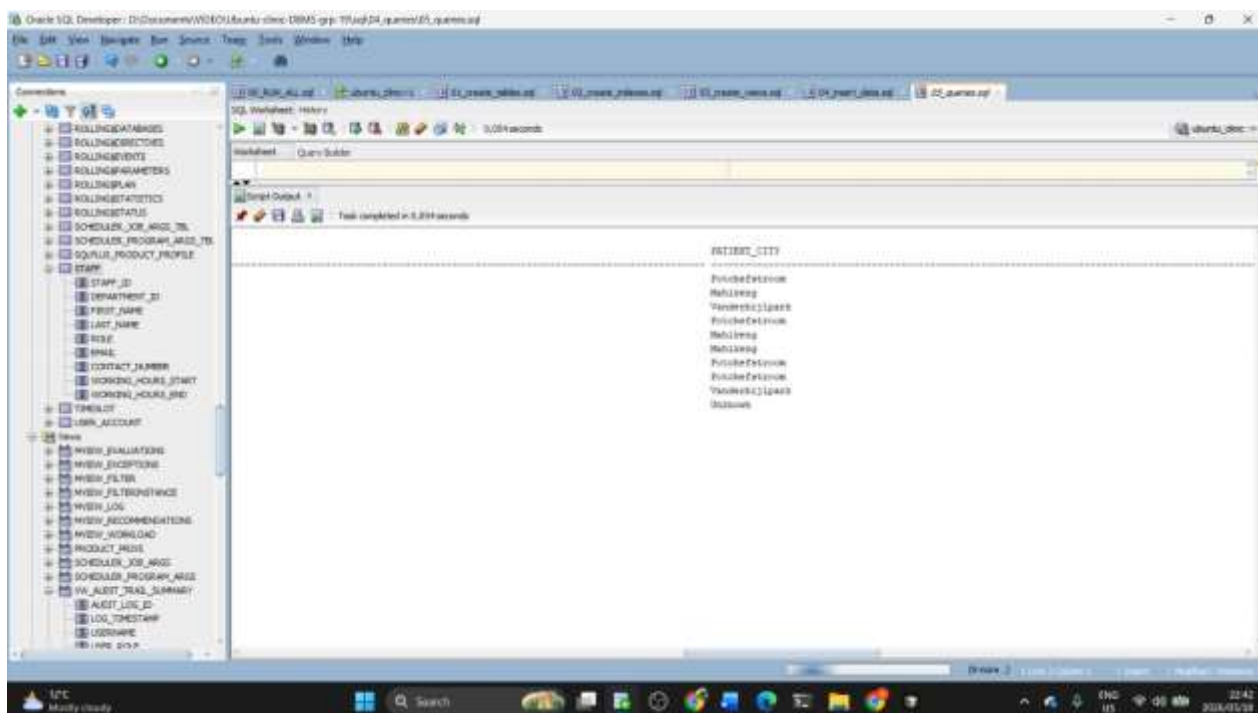


Appendix B: Populated Data



Screenshot showing successful data population in the PATIENT table with 10 rows selected.

Appendix C: Database Objects



Screenshots showing the STAFF table structure and database views.

Oracle SQL Developer: D:\Documents\VIDEO\ubuntu-clinic-DBMS-gp-19\sp34_queries\01_queries.sql

File Edit View Database Run Source Tools Help Window Help

Connections: 01_RUN_SQL.sql, 01_RUN_SQL.sql, 01_RUN_SQL.sql, 01_RUN_SQL.sql, 01_RUN_SQL.sql, 01_RUN_SQL.sql, 01_RUN_SQL.sql, 01_RUN_SQL.sql, 01_RUN_SQL.sql, 01_RUN_SQL.sql

SQL Worksheet: History

Worksheet: Query Editor

Script Output: Task completed in 0.004 seconds

APPOINTMENT_ID	APPOINTMENT_STATUS	BOOKING_TYPE	PRIORITY	SLOT_DATE	START_TIME	END_TIME	DOCTOR_NAME
1	COMPLETED	SICK	NORMAL	18/03/24	09:00	09:30	Haleed Mokone
2	COMPLETED	FOLLOW_UP	NORMAL	18/03/24	10:00	10:30	Thabo Elomisi
3	COMPLETED	SICK	URGENT	18/03/24	09:00	09:30	Alana Patel
4	NO_SHOW	SICK	NORMAL	17/03/24	11:00	11:30	Haleed Mokone
5	CANCELLED	WALK_IN	NORMAL	17/03/24	14:00	14:30	Thabo Elomisi
6	CONFIRMED	SICK	URGENT	19/03/24	08:00	08:30	Alana Patel
7	SCHEDULED	FOLLOW_UP	NORMAL	19/03/24	09:00	09:30	Haleed Mokone
8	SCHEDULED	FOLLOW_UP	NORMAL	18/03/24	10:00	10:30	Haleed Mokone
9	SCHEDULED	VIRTUAL_TRIAGE	NORMAL	20/03/24	09:10	09:20	Thabo Elomisi
10	SCHEDULED	SICK	NORMAL	20/03/24	14:00	14:30	Thabo Elomisi

100% Mostly cloudy

Search

22:42 2024/03/18